strategy requires more than adding logs around model calls. It begins with designing observability as a core architectural capability from the earliest stages of AI application development.

As a best practice, instrument the complete end-to-end workflow, not just the LLM invocation. In modern AI systems, quality depends on prompt construction, retrieval, embeddings, reranking, tool calls, guardrails, memory, model selection, and post-processing. Each step should generate structured telemetry so teams can reconstruct the full path from user input to final output. Distributed tracing using OpenTelemetry, OpenLLMetry, or OpenInference conventions provides a strong foundation because it allows AI-specific events to be correlated with application and infrastructure telemetry.

Second, define meaningful metrics across four dimensions: performance, cost, quality, and safety. Performance metrics should include latency, throughput, failure rate, retry count, timeout rate, and dependency response time. Cost metrics should track prompt tokens, completion tokens, embedding tokens, model cost, cache savings, and cost per successful task. Quality metrics should measure groundedness, relevance, factual consistency, citation accuracy, response completeness, and user satisfaction. Safety metrics should include harmful content detection, prompt injection attempts, data leakage indicators, refusal accuracy, and policy violations. Monitoring only infrastructure metrics gives an incomplete picture; AI systems require behavioural and semantic observability.

Another best practice is to connect online observability with offline evaluation. Production traces should be sampled and converted into evaluation datasets. These datasets can be used to compare prompt versions, model upgrades, retrieval strategies, and agent logic before deployment. Tools such as Langfuse, Arize Phoenix, TruLens, and MLflow can support this feedback loop by linking traces, experiments, evaluation scores, and model or prompt versions. This creates a disciplined improvement cycle where changes are validated with evidence rather than intuition.

Teams should also implement prompt and model versioning. Every production response should be traceable to a specific prompt template, model version, retrieval configuration, embedding model, and safety policy. This is essential for regression analysis. If answer quality drops after a rollout, teams must be able to identify exactly what changed. Canary releases, A/B testing, and automated evaluations should be used before fully deploying new prompts, models, or orchestration logic.

Data governance must be built into the observability pipeline. Prompts and responses may contain personally identifiable information, confidential business data, or regulated content. Therefore, teams should apply redaction, encryption, access control, retention policies, and environment separation. Sensitive payloads should be masked where possible, while still preserving enough metadata for debugging and evaluation.

Challenges and future direction

Despite rapid progress, AI observability is still an evolving discipline. Standards for semantic conventions, evaluation methods, and agent tracing are improving but not yet universally settled. Different frameworks emit different metadata, providers vary in what they expose, and quality measurement remains partly domain specific. Even so, the direction is clear. The industry is moving towards richer trace semantics, stronger evaluation integration, and more unified telemetry pipelines that can cover infrastructure, applications, and AI workflows together.

Observability for AI and LLM applications is no longer optional. It is the discipline that turns opaque, probabilistic systems into services that can be understood, trusted, and improved. By combining traces, metrics, logs, evaluations, and governance controls, organisations can diagnose failures faster, optimise cost, improve response quality, and deploy AI capabilities with greater confidence. Open source tools contribute an important piece of this ecosystem. Together, they help engineering teams build AI services that are not only intelligent, but also observable, accountable, and ready for production at scale. END

References

- *https://mlflow.org/ai-observability*
- *https://arize.com/blog/llm-observability-for-ai-agents-and-applications/*
- *https://blog.jetbrains.com/pycharm/2026/05/llm-evaluation-and-ai-observability-for-agent-monitoring/*
- *https://machinelearningmastery.com/llm-observability-tools-for-reliable-ai-applications/*
- *https://konghq.com/blog/learning-center/guide-to-ai-observability*
- *https://www.ibm.com/think/topics/llm-observability*

By: Dr Magesh Kasthuri and Dr Anand Nayyar

Dr Magesh Kasthuri is a PhD in artificial intelligence and the genetic algorithm. He currently works as a distinguished member of the technical staff (master) and chief architect at Wipro Ltd. This article expresses his views and not of the organisation he works in.

Dr Anand Nayyar is a PhD in wireless sensor networks and swarm intelligence. He works at Duy Tan University, Vietnam, and loves to explore open source technologies, IoT, cloud computing, deep learning, and cyber security.